Albatros Pipeline
(tentative)

Thomas Jamieson Bolder

July 3, 2025

Contents

Chapter 1

Introduction

This paper aims to be a resource of informal, high-level documentation on the data analysis pipeline of the ALBATROS low-frequency radio interferometry project. It should facilitate outside understanding of the project, whilst providing a solid framework for internal structuring. This paper is written by a third-year undergraduate in Mathematics and Physics, meaning the level of technical sophistication is reasonably surface-level. The author, although having contributed to the project, in no way claims full responsibility for the work outlined here.

Figure 1.1 outlines the general data flow of the overall pipeline.

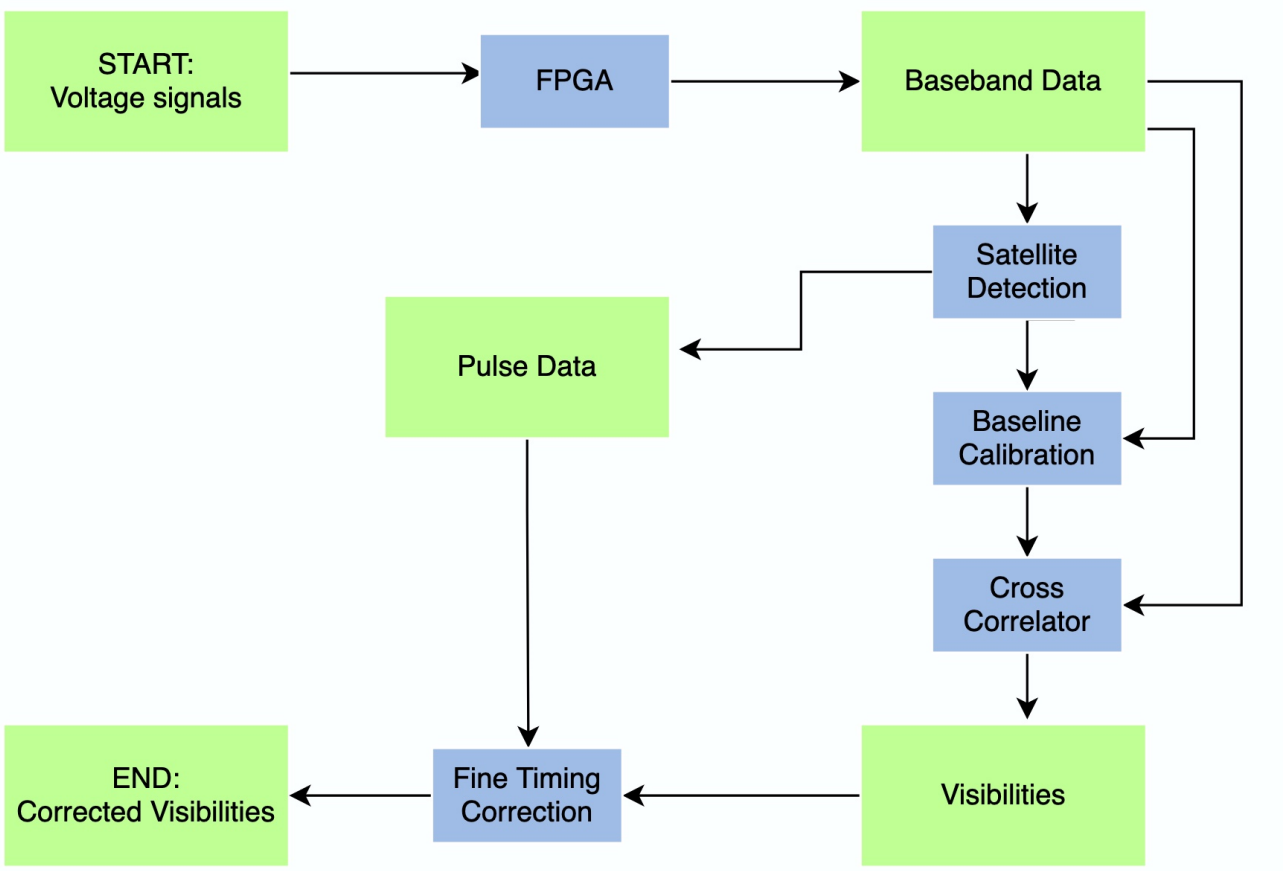


Figure 1.1: Overall pipeline data flow. Data is in green, whereas processes are in blue.

Chapter 2

The Antenna

Here are some things to discuss in this chapter. It's more of a firmware thing (which I am far from being an expert in), useful for some context into how we get our voltages initially.

- some details on how the antenna works
- polarizations and conventions
- the form of data it records
- clocks: the RPI clock and the nanosecond iterator. find some name for these guys.

Perhaps also discuss the antenna array we are using, as well as the baselines we consider.

2.1 Clocks

Each antenna has a human-time clock in units of unix time, which can drift significantly over the course of a day.

It also has a clock which makes a measurement every 4ns, which is much more reliable but still subject to some jitter.

Chapter 3

Channelization

Disuss how we get from voltage to Baseband data. Spectral window, and explain the choices we make.

Chapter 4

Baseband File System

This chapter is here to give an overview of (and hopefully in more polished versions) give some technical details into how we store and manipulate baseband data. Not super straightforward, and took me a while to understand.

- Baseband files are organized by UTC time. First the split is by 5 numbers (e.g. 17219), and then split again into raw files of length of about 45 seconds (in the MARS 1 case). They have a length of 10 numbers, corresponding to the UTC time at which the measurement of their data *starts*. These raw files then contain the FTed data for all our channels, in complex numbers.
- also note that there are two ways for us to store data, corresponding to different number of bits: 1 bit complex or 4 bit complex (corresponding to 2 bits total, or 8 bits total respectively).
- Since it's an insane amount of data, it is compressed. The way in which it is compressed is in an 8-digit binary, corresponding to either an integer or a complex number. unpacking into float complex 64 takes a bunch of time right (explain better too lazy to clean up right away) ('uint8' vs 'complex64')
- It may also LOSE DATA, so some spectra are missing. This isn't great, but there are functions to help remedy this. Make continuous and such are very valuable. Give brief overview, and perhaps link the function page itself.

Chapter 5

Timing

5.1 Framing the Objective

With an understanding of the collection and storage of baseband data, we can move on to the question of timekeeping. The primary issue we must overcome is the misalignment of the human-time clocks in each antenna. As mentioned in Chapter 2, each antenna has a drift-prone human-time clock. Every time we fetch baseband data at a certain point in unix time, we encounter some unknown offset between when we think it was measured and when the data was actually measured. This is problematic when computing visibilities, since extracting any useful interferometric information requires the two timestreams to be perfectly aligned. However, each antenna also has an internal nano-second iterator, meaning each spectrum (discrete collection of such nano-second iterations) is almost perfectly (for the purposes of this section) spaced. Thus, if we can find the exact offset in spectra between each antenna at a certain point, we know the exact offset for the entire day. Figure 5.1 illustrates this visually.

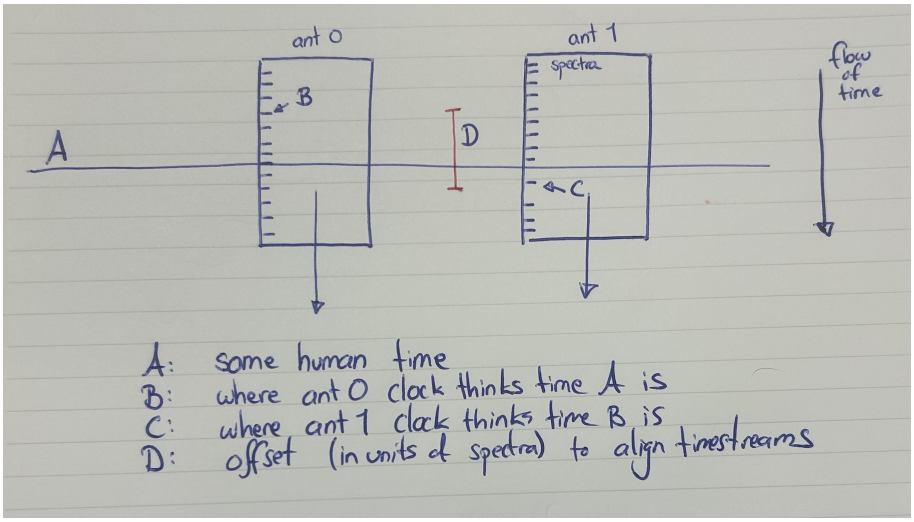


Figure 5.1: (Temporary!) Illustration of time offsets in Baseband data timestreams.

Note that in this case, the exact observer time loses its significance, since all we really care about is the alignment between timestreams. This game of finding the spectrum index offset between antenna timestreams is called "spectrum number alignment".

As a side note, since spectra have some finite duration (about 16 μs in our case), we cannot align the datastreams to an infinite level of precision, rather simply to the closest spectrum

number integer. However, it gives an excellent approximation of the clock offset, and will lead us into more precise corrections later in the analysis (see Chapter 7).

5.2 Satellite Passes

Spectrum number alignment relies on measuring some signal that is visible from both antenna. We require a source which: is bright to ensure high SNR, has a known position (for beamforming), and has some form of regularity for observation verification. The best candidates are low orbit satellites, since their signals are strong, their positions are well documented and highly accurate, and each satellite passes overhead approximately every 90 minutes.

Figure 5.2 Let us visualize what a satellite pass looks like for two antenna.

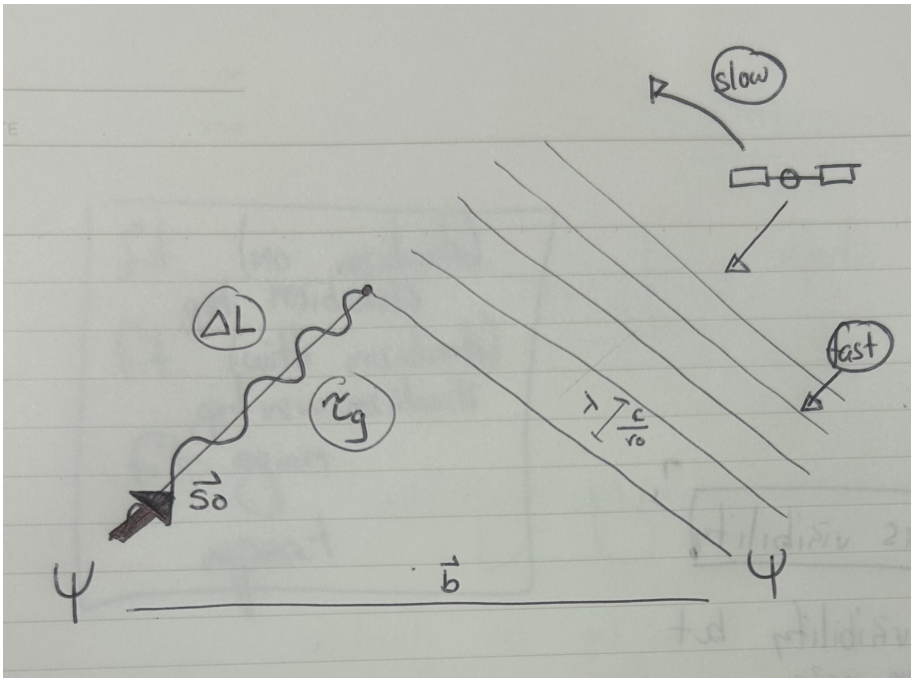


Figure 5.2: (Temporary!) Illustration of a satellite pass

Consider a very strong satellite signal at frequency channel ν_0, in direction $\vec{s_0}$. Let us assume that the incoming wave is flat, i.e. let us apply the flat sky approximation. In the interest of theoretical understanding, let us also assume the satellite signal is far brighter than anything else the antenna sees. Thus, we let the sky brightness function $I(\nu, \vec{s})$ be a delta function in the satellite's direction $\vec{s_0}$ and at its frequency ν_0. We make no assumption on the brightness function in other channels. Moreover, we also assume the antenna response is uniform.

Let's assume the electric field measured by Antenna 0 is $E_0(t) = a(t)e^{2\pi i\nu_0 t}$. Let's also assume that at a given time t, the total delay between arriving at Antenna 0 and Antenna 1 for any given wavefront is $\tau(t)$. Thus, the signal measured at Antenna 1 is $E_1(t) = E_0 e^{2\pi i\nu_0\tau(t)}$. This delay takes into account the datastream offset (τ_c), as well as a geometrically generated offset

(τ_g) we can see in Figure 5.2. The total delay, as expected, is given as the sum of geometric delay and clock delay: $\tau(t) = \tau_c(t) + \tau_g(t)$.

Our objective here is to find $\tau_c(t)$ given our knowledge of τ_g (easily derived from the satellite's position). The general idea is to remove the known geometric delay by applying a phase to one of the timestreams. Thus, we correct the second antenna timestream such that $E_1(t) = E_0 - e^{2\pi i \nu_0 \tau_g(t)}$ We beamform in the suspected direction of a satellite pulse, and try several time delays.

$$\begin{aligned}(E_0 * E_1)(\tau) &= \int_{t_0}^{t_0+t_{\text{int}}} E_0(t) E_1^*(t+\tau) dt, \\ &= \int_{t_0}^{t_0+t_{\text{int}}} E_0(t) \left[E_0(t+\tau) e^{-2\pi i \nu_0 \tau_g(t)}\right]^*, \\ &= \int_{t_0}^{t_0+t_{\text{int}}} E_0(t) \left[E_0(t+\tau_c)\right]^*\end{aligned} \tag{5.1}$$

This implies that the clock delay is found as the input of the maximum of the convolution of the two signals. This is a relatively rudimentary analysis of how this is done, a more detailed version is found in Mohan's notes on the coarse cross correlation.

We are assuming that the geometric time delay does not change significantly over the course of the convolution. This is true as the accumulation length t_{acc} is much smaller than the period of the phasor $e^{2\pi i \nu_0 \tau_g(t)}$. Also expressed as sayng the spectrum interval is smaller than a period of the fringe phase.

As a conceptual sidenote, when first approaching this derivation one can wonder: $\tau_g(t)$ is order 100ns, and a spectrum is about 16 μs, so how in the world does such a small shift make a difference? You remove add a phase which compensates for this path difference. This is easily computable given the TIME difference, the ANGLE difference is then just $e^{2i\pi f \tau_g}$ or something like that. So that lets you get a clean lineup between the two antenna datastreams. Once you get a clean lineup, I think you basically pretend that the antenna are "pointing in the same direction", i.e. are sensitive to signals from that exact direction (or at least with that same phase offset). Then can get a super nice SNR to get the offset of specnums. I think this is what's called a beamformed visibility.

Beware! In the previous section, we work in phase delay. Now, we identify the peak with the spectrum number offset, as you take the DFT to get the power spectrum with a time offset. WITH the above section, you use $\tau = \tau_g + \tau_c$ and actually remove the τ_g part. The coarse xcorr should give you the last part, the clock (τ_c) offset.

5.3 Practical Implementation

Must use a discrete coarse cross-correlation.

A large part of the efficacy of this method relies on accurate and reliable information on the satellites' exact coordinates at all points in time. This does two things. First, it allows us to

know what satellites are risen and when.

If we know that a satellite is in the field of view of the antenna, we say it is **risen**. The duration over which this satellite is risen is referred to as a **pass**. However, not all risen satellites are visible in the satellite data. If we can find the satellite in the correlation of the antenna data, we say it is **detected**, and the duration of time it is risen is a **pulse**, since we observe a peak in the data for these times. Our analysis therefore starts with a long list of risen satellites, with their pass times and satellite ID. We aim to detect these.

Secondly, the TLE files allow us to calculate the geometric delay between the antenna using this TLE file data. Applying the expected geometric offset of the satellite pulse to one of the antenna improves the signal strength received in the coarse cross correlation. Moreover, if there are multiple satellites present in the pass, applying the different expected geometric delays helps differentiate between them. Therefore, our analysis resumes by finding the antenna data for the approximate time of the pass taking one appropriately sized chunk from each antenna, and applying the appropriate phase to one of the steams before correlation.

We now have one chunk of data, at a time where we know the satellite should be visible, with the appropriate expected phase applied. Before continuing, we resolve a couple intricacies of this approach.

- well why does the geometric delay play such a large role if it's so much smaller (time delay) than the actual length of a spectrum number? Because it's a complex number, so a delay is just applying a phase change. Way more conceptual understanding to develop earlier to then have this make sense

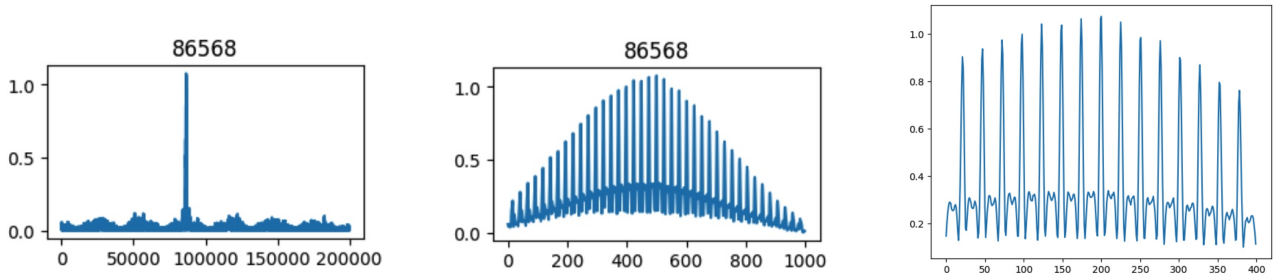
- if we don't know the time, how can you then know when to apply what offset and where? This is unresolved in my head, but I suspect that it doesn't matter too much, and that in fact just the ballpark changes the result quite a lot since the time it takes for the satellite to go through a full 2π phase change is way larger than the error in time. Might be wrong, and certainly worth looking into.

5.4 Analyzing Pulses

We now do a coarse cross-correlation for this chunk. We obtain this type of plot:

The convention is that 100'000 is a zero spectrum number offset. Clearly we see a peak at a specific correlation offset, which implies the initial spectrum number offset was wrong by that much, which implies the relative spectrum number offset is (initial - relative).

However, not all plots look like this. There are russian satellites and american satellites, and have reliable or unreliable peaks. Can differentiate between them using what we call a **reliability ratio**. A crude but effective way of measuring if a satellite pulse can confidently tell us the offset.



(a) Image 1 (b) Image 2 (c) Image 3

Figure 5.3: A usual American satellite pulse. Left is most zoomed out, right is most zoomed in. Can see the periodic peaks.

Talk about this for a bit, then move to next:

Important to record all pulses and their times for later visibility correction. Will get to that later.

Chapter 6

Baseline Calibration

6.1 Framework

At this point in the pipeline, the positional coordinates of each antenna are only known to approximate values. Since interferometry is heavily reliant on highly accurate baseline vectors, using such rough coordinates would yield unsatisfactory visibilities. Therefore, we must calibrate our antenna baselines as a preliminary step for further data analysis. As we will explore in more detail in the following sections, the calibration process itself is performed by minimizing phase residuals between observed and predicted satellite passes. Let us first visualize what our antenna array might look like with an example shown in Figure 6.1.

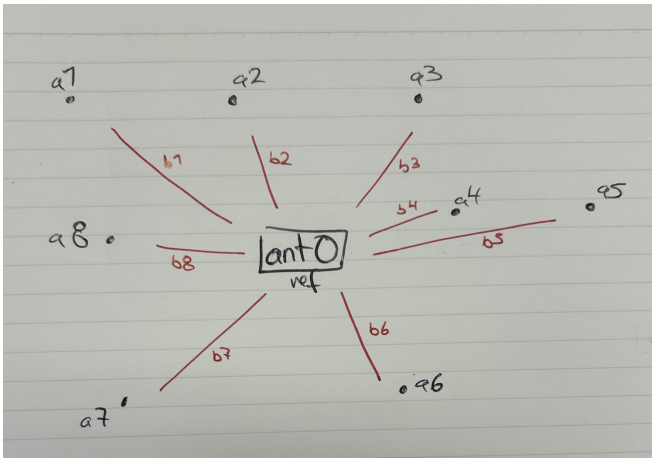


Figure 6.1: Example of Antenna Cluster. Reference Antenna need not be centrally positioned.

We initially define a reference antenna (also referred to as Antenna 0), which will keep its initial estimated coordinates throughout the fitting. All other antenna are labeled with natural numbers 1, 2, 3, and so on. We only consider baselines originating from the reference antenna, and label them by the name of their connecting non-reference antenna. For example, as seen in Figure 6.1, the baseline connecting Antenna 0 to Antenna 5 is labeled baseline 5.

Since we only require baseline vectors and not actual geodetic coordinates, a highly precise antenna position with respect to earth is of secondary importance. This allows us to fit each of these baselines individually and independently, using the non-reference coordinates as the only fitting parameter, determining the value which returns the best relative coordinates.

With the general layout in mind, we observe the overall Baseline Calibration data flow in Figure 6.2.

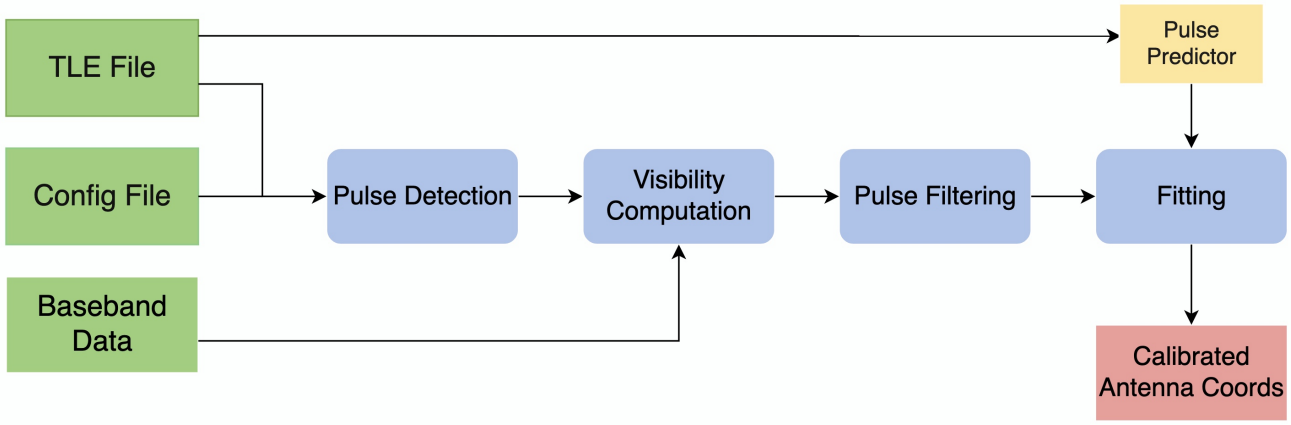


Figure 6.2: The Data flow is split into four main working modules (blue), with three primary inputs (green), one fitting predictor function (yellow), and one primary output (red).

Visibility Computation and filtering, the main observed data processing, are detailed in Section 6.2. Phase prediction and synthesis is discussed in Section 6.3. Finally, the fitting process and its complications is outlined in Section 6.4. Since this process is still in development, we explain the future outlook in Section 6.5.

6.2 Observed Pulses

6.2.1 Theoretical Pulse

Let us again consider a single two-dimensional satellite pulse (shown previously in Figure 5.2). We make assumptions of a flat sky, high satellite signal brightness, and uniform antenna response to simplify the theory. Recall that we performed a cross cross correlation and found the exact spectrum number offset. Thus, using the raw baseband data, if we correct for known clock offset τ_c and perform direct cross correlation, we will obtain complex visibilities with the geometric time delay τ_g encoded into them. This is derived below:

$$\begin{aligned} V_{0,1} &= \langle E_0(t)E_1(t)\rangle_{t_{\text{int}}}, \\ &= \langle E_0(t)E_0^*(t)e^{-2\pi i\nu_0\tau_g(t)}\rangle_{t_{\text{int}}}, \\ &= e^{-2\pi i\nu_0\tau_g}\langle E_0E_0^*\rangle_{t_{\text{int}}} \end{aligned} \tag{6.1}$$

Here we assume that the satellite moves slowly with respect to the integration time. Since E_0 correlated with itself is high, we expect a dominant presence of the $e^{-2\pi i\nu_0\tau_g}$ term even in a realistic, noisy sky.

Therefore, we consider $\tau_g(t)$ to be constant for a single visibility computation. Thus, we have a complex visibility which has the geometric delay encoded into it, so for many pulses we can unwrap this phase and obtain a phase plot over time.

6.2.2 Practical Pulse

Practically, we compute visibilities per chunk, blah blah

We use an accumulation length of $3 * 10^5$ spectra, with each spectrum having a period of approximately $16\mu s$, leaving us with a chunk period of approximately $0.5s$. Usable sections of pulses usually last several minutes, meaning they typically contain between 500 and 1000 chunks. i here is the index in the spectrum number, N is the accumulation length, i.e. length of chunk.

$$\begin{aligned} V_{0,1,i} &= \frac{1}{N}\sum_{i=0}^{N-1} E_0[i]E_1[i] \\ &= \frac{1}{N}\sum_{i=0}^{N-1} |E_0[i]|^2 \, e^{2\pi i \tau_g[i]} \end{aligned} \tag{6.2}$$

Figure 6.3 shows the angle with time for the visibility of all channels during a pulse. Can clearly see how the angle of the geometric time delay dominates channels index 2 and 3.

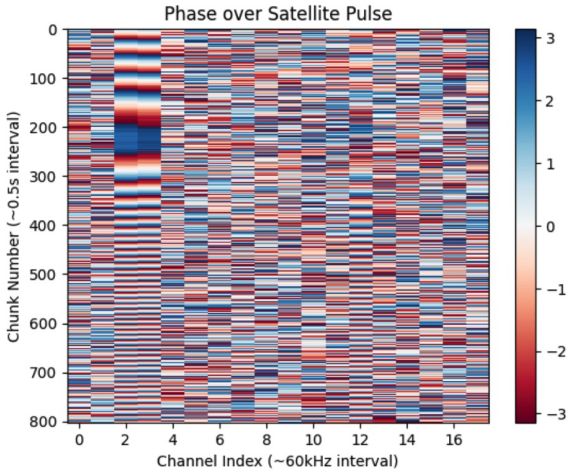


Figure 6.3: Wrapped visibility phase. The satellite pulse is visible in channels 2 and 3.

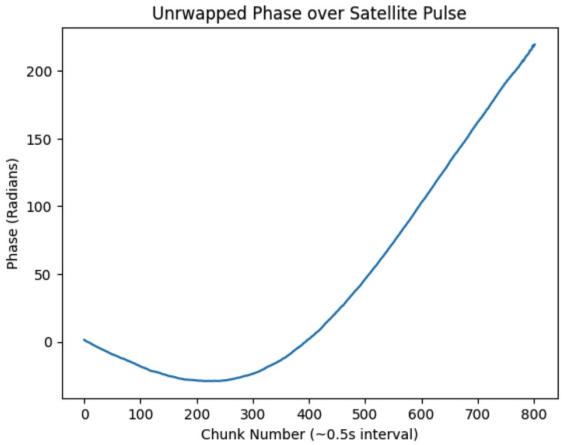


Figure 6.4: Unwrapped phase for channel 2 over the entire pulse.

To obtain the observed phase for each satellite pulse, we must use the satellite detection script (outlined in Chapter 5) to get the pulse times, channels, and the spectrum number offsets. Spectrum number, of course, being our proxy for time. We correct for this as our clock offset τ_c. Once this information is recorded and implemented, we can correlate Baseband data from the reference and non-reference antenna to obtain raw visibilities. We do this for each pulse, for each day of data, and for each baseline.

The unwrapped phase is the final product of the visibility computation, and will be used as our observed data for the eventual coordinate fit. For each baseline, the total observed data is compromised of several such unwrapped pulse phase curves.

6.2.3 Filtering

To obtain a reasonably good plot, we must often massage the data into a better form. For example, we interpolate over missing chunks (directly in the visibilities), and we must also sometimes cut the pulse from the front or the back to remove noisy or aliased (check if this is actually aliasing when going fast, but either way) parts.

Aim is to construct an automatic filter and cut for these, since the data is not fixable, and it's inherently not good at some points (I believe, again double check)

- there is some data missing, happens far more often with 1bit data. we interpolate over bad data directly in the visibility. done in rectangular and then recombined. show an example plot

- we must then also filter through the bad UNWRAPPED visibilities phases. the current model fits for unwrapped phases, so want to ensure we have a continuous and clear unwrapped phase plot.

- in the future, implement one of, or both of, automatic slicing and interpolation over bad data. This would speed up the data pipeline and make more pulses usable for a longer time.

6.3 Predicted Phase

In order to correctly fit the non-reference antenna's coordinates, we must construct a function which accurately predicts the satellite pulse phase over time. Its only free parameter must be the non-reference antenna's coordinates. Once we have constructed this function, we can evaluate the 'goodness of fit' of the initial guess coordinates as a preliminary test. This can be seen in Figures 6.5 and 6.11.

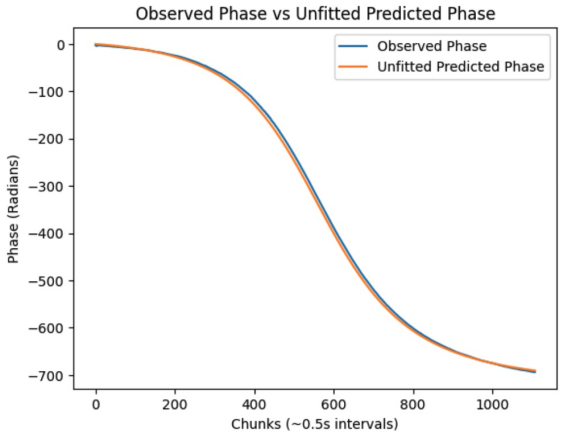


Figure 6.5: Unfitted predicted phase against observed phase. Visible discrepancy.

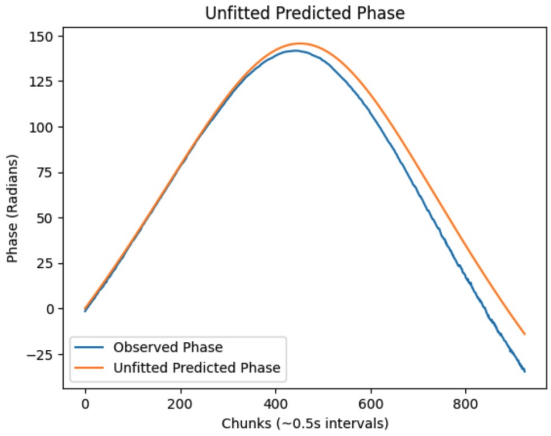


Figure 6.6: Different predicted phase, now plotted alongside observed phase. More significant discrepancies.

Some pulses will initially be rather well fit, such as Figure 6.5, whereas others will leave more to be desired, such as Figure 6.11. These illustrate the necessity of fitting for multiple satellite passes simultaneously, as different satellites will have different trajectories in the sky, and thus will favor different fits.

6.4 Fitting

6.4.1 Moving Parts

There are several things we need to take into account during the fitting process.

First of all, we must take into account the thermal noise on each pulse. The residuals of each pulse must be weighted using the noise of the pulse, to ensure that noisy pulses are considered less.

Secondly, we must ensure that our various pulses cover several different directions in the sky. A baseline is a single line, so passes have to move over at different angles to properly calibrate it. DEVELOP?

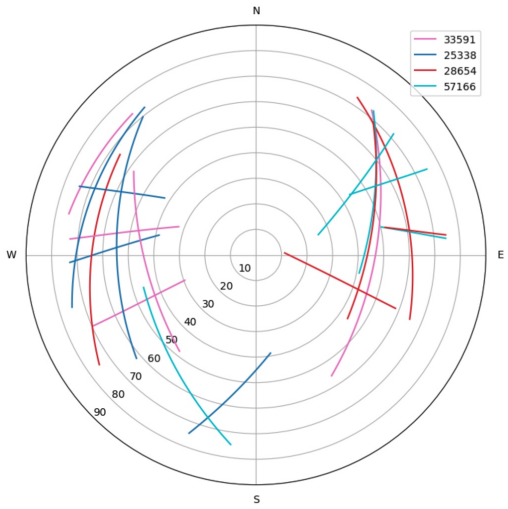


Figure 6.7: Example of a satellite pulse plot, in direction over the sky

Finally, we must take into account offset between predicted phases and observed phases. We assume that when we predict a phase, it is accurate in human observer time. We also assume (and pray) that the visibility spectrum number offset correctly aligns the two clocks of the two antenna. This is done in such a way that leaves Antenna 0's clock unchanged, and shifts the spectra of Antenna 1 (which makes sense or else it would be super confusing with multiple antenna). But we know that there is significant clock drift in the human time (RPI) clock of the Antenna. Since each pulse is accessed separately using human time, each pulse has the risk of having a time offset associated to it. So we must implement a per-pulse time offset variable that can correct for this.

Therefore, the fitting process has three separate weighting methods, alongside three separate fitting types. These are modular; any fitting method can be used with any of the weighting methods.

6.4.2 Weighting Methods

6.4.3 Fitting Methods

We do several types of fits. First is only with coordinates.

Obtain residuals from each pulse, concatenate into one large array, and least squares fit with your predictor function. Only fitting parameter is the coordinate for the predicted phase. This is done simultaneously for all considered pulses.

This gives us one possible fit for one baseline.

Figure 6.8 shows six pulses with their fitted phases.

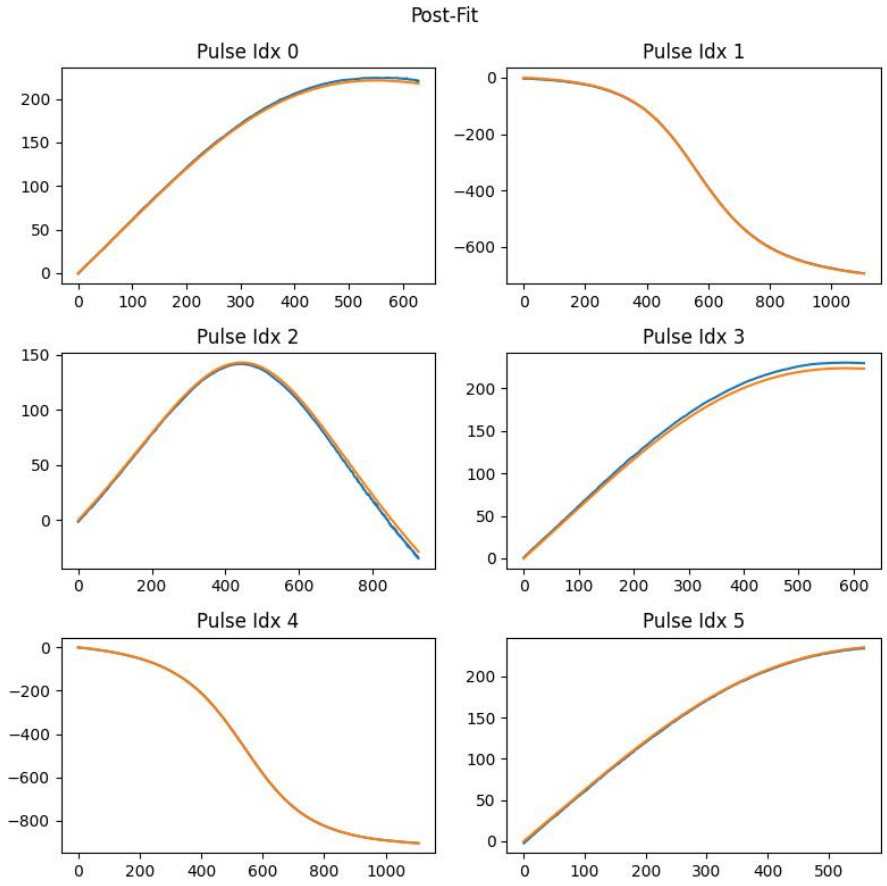


Figure 6.8: Fitted with the TRF method. In this run, TRF and LM gave negligibly different results.

We can see how Pulse Index 1 (seen unfitted in Figure **??**) has very close agreement between observed and predicted phases. This is further exemplified in the residuals plots, seen in Figure 6.9.

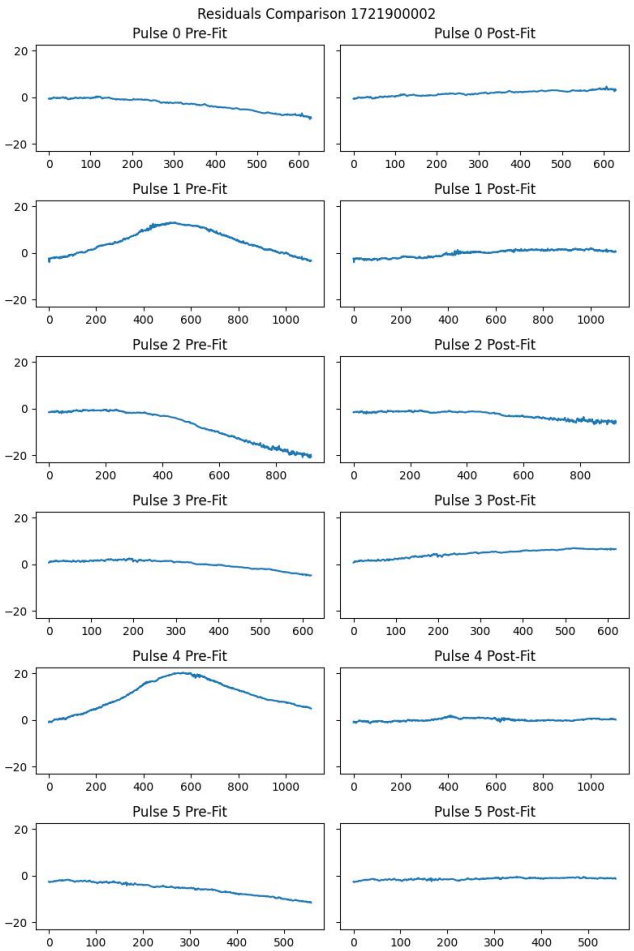


Figure 6.9: All pulses with residuals before and after fitting. Again with TRF algorithm.

Secondly, we have a per-pulse time offset fit. The reason for this is the time discrepancy (in human observer time) between getting

6.4.4 Testing Methods

To ensure we have found a valid coordinate, we must conduct some testing. Naturally, this means varying some fixed parameters and ensuring our coordinates converge to the same values.

First, we must vary the initial guess for the non-reference antenna. This ensures that the fit is not stuck in a local minimum. In testing, several randomly generated nearby coordinates are also tested, and their results are compared. For a coordinate to be deemed valid, they must all return negligible differences in coordinates.

Second, we utilize two fitting methods as a safety check. We use both the 'Trust Region Reflective' and 'Levenberg-Marquardt' algorithms of least squared fitting. For a coordinate to be deemed valid, both fit method must yield a negligible differences.

Third, we fit over multiple sets of 5-6 pulses, and vary combination of satellites. This ensures there is no bias in pulse direction, satellite type, or time of day. A valid coordinate must, as for other variables, remain consistent across pulses.

These tests are conducted for each baseline, using the single baseline coordinate calibration script.

6.5 Limitations and Future Development

As of the writing of this section, there are several technical limitations causing difficulties in the development of a streamlined, generalized coordinate calibration script.

The main issue is the inconsistency in observed phase plots. This is the result of low SNR satellite passes (which are detected but not good), unwrapping bugs, or missing data. This means pulses must be combed through manually before being passed through the calibration script, significantly slowing analysis. Figures 6.10 and **??** provide an example of such a pulse.

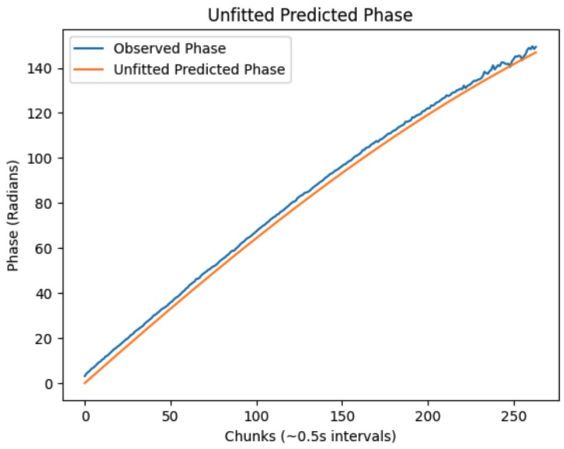


Figure 6.10: Unfitted predicted phase against observed phase. Visible discrepancy.

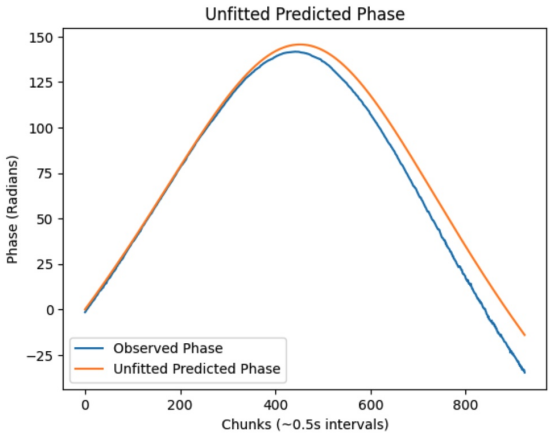


Figure 6.11: Different predicted phase, now plotted alongside observed phase. More significant discrepancies.

This issue will be resolved before making a generalized script that can fit and test coordinates multiple baselines automatically. This would be the optimal case, and would significantly facilitate any future project's baseline optimization.

Chapter 7

Fine Timing